Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Counter Reference: What Is Counted and How to Read It

gem5 TimingSimpleCPU · RISC-V (RV64GC) · SPEC CPU2017 · metric schema 10

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Assistant Prof. Dr. İsmail Aktürk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

12 November 2026

1. What is measured

A spill is detected *dynamically*, from the runtime effective addresses of the committed loads and stores alone. No compiler information, no register name and no base-register test takes part. A store to the process stack is a *spill store*; a later load that reads a byte it wrote is a *spill load*; a spill store read back at least once is a *spill event*.

This is an operational definition: *runtime-detected stack store–load reuse*, or spill-like memory traffic. Register-allocator spills, callee-saved register saves and restores (return address included), memory-resident locals and stack-passed arguments all meet it and cannot be told apart from addresses alone. The counts are therefore not counts of compiler-generated spill instructions.

Counting is confined to the region of interest (ROI), marked by `m5_work_begin` and `m5_work_end` in the benchmark sources. In a SimPoint run the ROI is the simulated interval after the warmup. Stores before the ROI, the warmup included, are not tracked, so a reload of one never pairs.

Table 1. Counter names. Ratios are not counters: the report scripts compute them. gem5's own statistics keep their names.

Name	Scope
`total_`*name*	The whole simulation; for a SimPoint estimate, the whole program (from the qemu profile).
`roi_`*name*	Only between the ROI markers; SimPoint: the interval.
`diag_`*name*	A technical diagnostic: checks the simulation and the matching, not a result.
`roi_`p`_{l1,l2}_{hits,misses}`	The L1D or L2 outcome of the accesses of population $p \in$ {`spill_store`, `spill_event`, `spill_load`}.

2. When a store or a load counts

Table 2. The conditions, checked when the instruction commits. S: a store, L: a load.

	Condition	Why
S1	An ordinary store: not a store-conditional (SC), not an atomic memory operation (AMO)	Atomics are out of scope
S2	It commits inside the ROI	Measurement window
S3	Its effective address lies in the process stack, `stackMin` $\leq$ addr $<$ `stackBase`	Spill slots live on the stack
S4	It completes without a fault	A faulted attempt is discarded; the re-executed store counts once
L1	An ordinary load: not a load-reserved (LR)	As S1
L2	It commits inside the ROI	As S2
L3	Its effective address lies in the process stack	As S3
L4	Its start byte was last written by a store meeting S1–S4. The newest writer owns each byte, so after a partial overwrite the newer store owns the overlapped bytes; the widths may differ	The value read was produced by that store
L5	The load is later than that store	Written before read
L6	That store still owns the byte when the load commits	Not overwritten in between
L7	It completes without a fault	As S4

A store meeting S1–S4 is counted in `roi_spill_store`, read back or not. A load meeting L1–L7 is counted in `roi_spill_load`; the first such load of a store also counts that store once in `roi_spill_event`. A successful SC or AMO still owns the bytes it wrote, so a later load does not pair through it with an older store; a failed SC wrote nothing and changes nothing. The stack pointer is read only for the stack-release diagnostic; it never decides whether an access is a spill.

3. Detector counters

The detector writes these to `riscv_spill_stats.txt`: one ROI block per ROI and one global block.

Table 3. The counters the detector writes.

Counter	Counted when	How to read it
`metric_schema`	Constant, 10 here.	Which spill definition produced the result. Results of schema 9 and earlier use other definitions and must not be mixed in.
`total_insts`	Every committed instruction of the simulation.	Length of the whole program; the reference for `roi/total`.
`total_load, total_store`	Every committed load or store.	Load and store counts of the whole program.
`roi_total_insts`	Every instruction committed inside the ROI (the markers excluded).	Size of the measured region: the denominator of every ROI ratio.
`roi_load, roi_store`	Loads and stores committed inside the ROI.	All of them, spill or not. LR counts as a load and SC as a store; an AMO is neither.
`roi_spill_store`	A store meeting S1–S4, read back or not.	How often the program wrote a value to the stack: spills, callee saves, stack locals together.
`roi_spill_event`	A spill store read back at least once: one per store, at its first spill load.	How many of those stack writes were actually used again. Never above `roi_spill_store` or `roi_spill_load`.

Counter	Counted when	How to read it
`roi_spill_load`	A load meeting L1–L7. Repeats count.	How often the program read a stack-written value back. A slot read three times counts three, so it can exceed `roi_spill_event`.
`roi_`*p*`_l1_hits, roi_`*p*`_l1_misses`	L1D hits and misses of population *p*'s accesses.	How much of that traffic L1D served, and how much went down to L2.
`roi_`*p*`_l2_hits, roi_`*p*`_l2_misses`	L2 hits and misses of population *p*'s L1D misses.	The misses went on to DRAM: the most expensive accesses.
`roi_`*p*`_split_insts`	Instructions of *p* whose access crosses a cache-line boundary; each makes two accesses.	Why L1 accesses can exceed the instruction count. Normally small.
`roi_spill_event_*`	The store side of each event: the store's own outcome when it executed, added once at its first reload.	"How did the stores that were read back fare in the cache?"
`diag_store_map_evicted`	Store-table entries dropped at the size cap (2^{20} entries).	Must be 0. Otherwise some pairs were lost and the spill counts are too low.
`diag_load_after_stack_release`	Spill loads of a byte that lay below `sp` at some point after its store: the stack there had been released.	A suspicious pair: another frame may have reused the address. It does not change the main counters.
`diag_event_after_stack_release`	Spill events whose first spill load is such a load.	The same suspicion on the event side.

4. gem5 statistics the reports read

Only the ROI block of `stats.txt` is read. The statistics are dumped and reset at the ROI boundaries.

Table 4. gem5 statistics used. L1D is `system.cpu.dcache`, L2 is `system.l2`.

Statistic	Used in	How to read it
`L1D demandHits, demandMisses, demandAccesses (::total)`	`L1D_miss`	Overall success of L1D in this configuration: every load and store, spill or not.
`L1D ReadReq.misses, ReadReq.accesses`	`other_ld_L1miss`	Ordinary loads. Removing the spill loads leaves the other loads.
`L1D WriteReq.misses, WriteReq.accesses`	`other_st_L1miss`	Ordinary stores. Removing the spill stores leaves the other stores.
`L2 demandHits, demandMisses`	`L2_local_miss`	Every request reaching L2: the L1D *and* L1I misses.
`L1D and L2 stackStoreHits, stackStoreMisses`	Cross-check	Added by this project. The name is historical; the values must equal `roi_spill_store_l1/l2_*`.
`L1D and L2 spillLoadHits, spillLoadMisses`	Cross-check	Added by this project; must equal `roi_spill_load_l1/l2_*`. If they are equal, the detector and the caches saw the same accesses.
`simInsts, numLoadInsts, numStoreInsts`	Tests only	gem5's ROI block covers the detector's ROI: `simInsts` = `roi_total_insts` +1, the end marker.

LR, SC and AMO accesses reach the cache as other request types, so they are in neither `ReadReq` nor `WriteReq`, and so in no "other" rate.

5. Values the reports compute

Table 5. Derived values.

Name	Formula	How to read it
`roi_spill_insts`	`roi_spill_load` + `roi_spill_store`	All spill-like instructions.
`roi_total_mem`	`roi_load` + `roi_store`	All memory instructions of the ROI.
`roi_unread_spill_store`	`roi_spill_store` − `roi_spill_event`	Stack writes never read back inside the ROI: overwritten first, dead, or still unread when the ROI ended.
`roi_non_spill_store`	`roi_store` − `roi_spill_store`	Stores outside the stack (heap, globals), and SC.

6. Program ratios

They do not depend on the cache, so all five configurations show the same values. Every ratio is a percentage.

Table 6. Program ratios.

Ratio	Formula (×100)	How to read it
`roi/total`	`roi_total_insts` / `total_insts`	How much of the program was measured. Full runs: 95–100%; low values mean the ROI may not represent the program.
`load/roi`	`roi_load` / `roi_total_insts`	Read intensity of the program.
`store/roi`	`roi_store` / `roi_total_insts`	Write intensity of the program.
`spill/roi`	(`roi_spill_load` + `roi_spill_store`) / `roi_total_insts`	**The main metric**: the share of instructions that write a value to the stack or read one back. High: register pressure, or many calls.
`spill_load/load`	`roi_spill_load` / `roi_load`	Share of the loads that read a stack-written value back.
`spill_store/store`	`roi_spill_store` / `roi_store`	Share of the stores that go to the stack, read back or not.
`spill_event/store`	`roi_spill_event` / `roi_store`	Share of the stores that write a value which is later read back.
`spill_event/ spill_store`	`roi_spill_event` / `roi_spill_store`	Reuse rate of stack writes. Near 100%: almost every stack write is used again; low: many stack writes are never read.
`spill/mem`	(`roi_spill_load` + `roi_spill_store`) / (`roi_load` + `roi_store`)	Share of the memory traffic that is spill-like. Unlike `spill/roi`, the denominator holds memory instructions only.

7. Cache miss rates

These depend on the configuration: the program and its instructions are the same, only the hits and misses change. An *access* is one request of a load or store to a cache (a cross-line instruction makes two); a *hit* ends at that level; a *miss* goes one level down (L1D → L2 → DRAM). The caches are write-allocate: a store miss first fetches the line. An L2 rate is *local*: its denominator is the L1D misses of that kind.

Table 7. Miss rates, in percent (×100). p accesses = `roi_`p`_l1_hits` + `roi_`p`_l1_misses`; likewise for L2.

Rate	Formula	How to read it
`L1D_miss`	L1D demand misses / L1D demand accesses	Overall L1D miss rate; the basic value to compare configurations.
`spill_ld_L1miss`	spill-load L1 misses / spill-load L1 accesses	Usually low: the slot was written shortly before, so its line is often still in L1D.

Rate	Formula	How to read it
`other_ld_L1miss`	(ReadReq misses − spill-load L1 misses) / (ReadReq accesses − spill-load L1 accesses)	**Read next to `spill_ld_L1miss`**: do spill loads miss less or more than ordinary loads?
`spill_st_L1miss`	spill-store L1 misses / spill-store L1 accesses	Miss rate of all stack writes.
`spill_event_L1miss`	spill-event L1 misses / spill-event L1 accesses	Miss rate of the stack writes that were read back.
`unread_st_L1miss`	(spill-store − spill-event L1 misses) / (spill-store − spill-event L1 accesses)	Miss rate of the stack writes never read back.
`other_st_L1miss`	(WriteReq misses − spill-store L1 misses) / (WriteReq accesses − spill-store L1 accesses)	Ordinary stores; compare with `spill_st_L1miss`.
`L2_local_miss`	L2 demand misses / L2 demand accesses	Share of the requests reaching L2 (L1D and L1I misses) that went on to DRAM.
`spill_ld_L2local`	spill-load L2 misses / spill-load L2 accesses	Of the spill loads that missed L1D, the share that missed L2 too.
`spill_st_L2local`	spill-store L2 misses / spill-store L2 accesses	The same for all stack writes.
`spill_event_L2local`	spill-event L2 misses / spill-event L2 accesses	The same for the stack writes read back.
`unread_st_L2local`	(spill-store − spill-event L2 misses) / (spill-store − spill-event L2 accesses)	The same for the stack writes never read back.

The denominators of the L2 local rates can be tiny (a few accesses can give 66.7%). Read them together with the absolute counts.

8. Combining results

Table 8. How several results become one number.

Where	Formula	How to read it
SimPoint estimate	$C = N \cdot \sum_i w_i\, c_i/n_i \,/\, \sum_i w_i$: N the ROI length from the qemu profile, n_i and c_i the instructions and the counter of point i, w_i its weight	A ref result is a weighted estimate, not a full run; its ratios are weighted per-instruction ratios.
pooled	$\sum$ numerators / $\sum$ denominators over all completed runs	Large runs dominate: the share of the total traffic.
mean	Average of the per-benchmark values	Every benchmark weighs the same: the typical benchmark.

9. Identities every result satisfies

The test suite asserts these, and every completed result is checked against them:

$$\texttt{roi_spill_event} \le \texttt{roi_spill_store} \le \texttt{roi_store}$$

$$\texttt{roi_spill_event} \le \texttt{roi_spill_load} \le \texttt{roi_load}$$

$$\texttt{roi_}p\texttt{_l1_hits} + \texttt{roi_}p\texttt{_l1_misses} = \text{count of } p + \texttt{roi_}p\texttt{_split_insts}$$

$$\texttt{roi_}p\texttt{_l2_hits} + \texttt{roi_}p\texttt{_l2_misses} = \texttt{roi_}p\texttt{_l1_misses}$$

$$\texttt{roi_spill_event_*} \le \texttt{roi_spill_store_*}$$

$$\text{gem5 } \texttt{stackStore*} = \texttt{roi_spill_store_*}, \quad \text{gem5 } \texttt{spillLoad*} = \texttt{roi_spill_load_*}$$

10. Worked example

A. Which instruction counts where

The fragment runs inside the ROI; `a2` and `a3` point to the heap.

Table 9. Contribution of each instruction (empty: no change).

Instruction	store	load	spill st.	event	spill ld.	Why
`addi sp, sp, -32`						Not a memory instruction
`sd a0, 16(sp)`	+1		+1			Ordinary store to the stack
`sd a1, 8(sp)`	+1		+1			Stack store; never read: unread
`sw t0, 0(a2)`	+1					Heap store: a non-spill store
`ld t1, 0(a3)`		+1				Heap load: an other load
`ld a0, 16(sp)`		+1		+1	+1	Owner is the first `sd`; its first reload: an event
`ld a2, 16(sp)`		+1			+1	Same store read again: no new event
`amoadd.w t2, t3, (a4)`						Atomic: neither load nor store
`addi sp, sp, 32`						Not a memory instruction
Total	3	3	2	1	2	`roi_total_insts` = 9

Derived: `roi_unread_spill_store` $= 2-1=1$ (the slot `8(sp)`), `roi_non_spill_store` $= 3-2=1$ (the heap store).

B. From counters to ratios

Assume a run reports `total_insts` = 1000, `total_load` = 300, `total_store` = 150; `roi_total_insts` = 800, `roi_load` = 240, `roi_store` = 120; `roi_spill_store` = 60, `roi_spill_event` = 45, `roi_spill_load` = 70.

Table 10. Program ratios of the example.

Ratio	Calculation	Value	Reading
`roi/total`	800 / 1000	80.00%	80% of the program was measured
`spill/roi`	(70 + 60) / 800	16.25%	About 16 of every 100 instructions write to or read back from the stack
`spill_load/load`	70 / 240	29.17%	Nearly a third of the loads are stack read-backs
`spill_store/store`	60 / 120	50.00%	Half of the stores go to the stack
`spill_event/store`	45 / 120	37.50%	37.5% of the stores write a value that is read later
`spill_event/spill_store`	45 / 60	75.00%	Three quarters of the stack writes are used again; 15 never are
`spill/mem`	130 / 360	36.11%	About a third of the memory traffic is spill-like

For the cache, assume no cross-line accesses and these outcomes: spill stores 54 L1 hits and 6 misses, of which the spill events 42 and 3; spill loads 68 and 2; L1D `ReadReq` 240 accesses and 30 misses, `WriteReq` 120 and 18; at L2, spill stores 4 hits and 2 misses (events 3 and 0), spill loads 1 and 1; L2 in total 60 accesses (48 from L1D,

12 from L1I) and 15 misses.

Table 11. Miss rates of the example.

Rate	Calculation	Value	Reading
`spill_ld_L1miss`	2 / (68 + 2)	2.86%	Spill loads almost always hit ...
`other_ld_L1miss`	(30 − 2) / (240 − 70)	16.47%	... ordinary loads miss far more: spill traffic is cache friendly
`spill_st_L1miss`	6 / (54 + 6)	10.00%	All stack writes
`spill_event_L1miss`	3 / (42 + 3)	6.67%	Stack writes read back
`unread_st_L1miss`	(6 − 3) / (60 − 45)	20.00%	Stack writes never read miss more
`other_st_L1miss`	(18 − 6) / (120 − 60)	20.00%	Ordinary stores
`L1D_miss`	48 / 360	13.33%	All L1D accesses
`L2_local_miss`	15 / 60	25.00%	All L2 accesses
`spill_ld_L2local`	1 / (1 + 1)	50.00%	Denominator 2: too few to interpret
`unread_st_L2local`	(2 − 0) / (6 − 3)	66.67%	Denominator 3: too few to interpret

Checks: $45 \leq 60 \leq 120$, $45 \leq 70 \leq 240$, $54 + 6 = 60$ and $4 + 2 = 6$ hold. At L1D gem5 must report `stackStoreHits` = 54, `stackStoreMisses` = 6, `spillLoadHits` = 68, `spillLoadMisses` = 2, and at L2 4, 2, 1 and 1.

11. Implementation and verification

Component	Source
Rules and counters	`src/cpu/simple/spill_detector.{hh,cc}`
Commit hook, request tags, LR/SC/AMO handling	`src/cpu/simple/timing.cc`
SimPoint ROI (the interval)	`configs/common/Simulation.py`
Per-population cache counters	`src/mem/cache/base.{hh,cc}`
Ratios and miss rates	`cache_scripts/cache_status.py`
SimPoint estimates	`cache_scripts/simpoint_combine.py`

The rules are covered by 27 hand-verified RISC-V assembly test cases in `tests/spill_test/`: partial overwrites, atomics, a SimPoint interval, frame reuse and stack growth among them. They assert every identity above and that the gem5 cache statistics and the detector counters agree at both cache levels. On 12 November 2026 every completed schema-10 result of the campaign, 170 test runs and 3405 SimPoint points, satisfied every identity; `diag_store_map_evicted` was 0 everywhere, and `diag_load_after_stack_release` was 0.05% of the spill loads.

12. Örnek (Türkçe)

Bu bölüm, 10. bölümdeki örneğin Türkçe anlatımıdır.

A. Hangi komut hangi sayacı artırır?

Kod parçası ROI içinde çalışıyor; `a2` ve `a3` heap'i gösteriyor.

Table 12. Her komutun sayaçlara katkısı (boş: değişmez).

Komut	store	load	spill st.	event	spill ld.	Neden
`addi sp, sp, -32`						Bellek komutu değil
`sd a0, 16(sp)`	+1		+1			Stack'e normal store
`sd a1, 8(sp)`	+1		+1			Stack'e store; hiç okunmayacak: unread
`sw t0, 0(a2)`	+1					Heap'e yazıyor: non-spill store
`ld t1, 0(a3)`		+1				Heap'ten okuyor: other load
`ld a0, 16(sp)`		+1		+1	+1	Byte'ın sahibi ilk `sd`; ilk reload olduğu için event
`ld a2, 16(sp)`		+1			+1	Aynı store'un ikinci okuması: yeni event yok
`amoadd.w t2, t3, (a4)`						Atomic: hiçbir load/store sayacına girmez
`addi sp, sp, 32`						Bellek komutu değil
Toplam	3	3	2	1	2	`roi_total_insts` = 9

Türetilenler: `roi_unread_spill_store` = 2 − 1 = 1 (`8(sp)` slot'u), `roi_non_spill_store` = 3 − 2 = 1 (heap store'u).

B. Sayılarla oranlar

Bir koşunun şu sonuçları verdiğini düşünelim:

- Tüm program: `total_insts` = 1000, `total_load` = 300, `total_store` = 150
- ROI: `roi_total_insts` = 800, `roi_load` = 240, `roi_store` = 120
- Spill sayaçları: `roi_spill_store` = 60, `roi_spill_event` = 45, `roi_spill_load` = 70

Table 13. Örneğin program oranları.

Oran	Hesap	Sonuç	Yorum
`roi/total`	800 / 1000	%80,00	Programın %80'ini ölçtük
`spill/roi`	(70 + 60) / 800	**%16,25**	Her 100 komuttan yaklaşık 16'sı stack'e yazma ya da stack'ten geri okuma
`spill_load/load`	70 / 240	%29,17	Load'ların yaklaşık üçte biri stack'ten geri okuma
`spill_store/store`	60 / 120	%50,00	Store'ların yarısı stack'e gidiyor
`spill_event/store`	45 / 120	%37,50	Store'ların %37,5'i stack'e yazılıp sonra okunan değer
`spill_event/spill_store`	45 / 60	%75,00	Stack'e yazılanların dörtte üçü geri kullanılmış; 15'i (unread) hiç okunmamış
`spill/mem`	130 / 360	%36,11	Bellek trafiğinin yaklaşık üçte biri spill-like

Cache tarafı için cache line sınırını aşan access olmadığını (`split_insts` = 0) ve şu sonuçları varsayalım:

- Spill store'lar: L1'de 54 hit, 6 miss; bunlardan spill event olanlar 42 hit, 3 miss
- Spill load'lar: L1'de 68 hit, 2 miss
- L1D `ReadReq`: 240 access, 30 miss; `WriteReq`: 120 access, 18 miss
- L2: spill store'lar 4 hit, 2 miss (event'ler 3 hit, 0 miss); spill load'lar 1 hit, 1 miss
- L2 toplamı: 60 access (48'i L1D'den, 12'si L1I'dan), 15 miss

Table 14. Örneğin miss oranları.

Oran	Hesap	Sonuç	Yorum
`spill_ld_L1miss`	2 / (68 + 2)	**%2,86**	Spill load'lar neredeyse hep hit ediyor …
`other_ld_L1miss`	(30 − 2) / (240 − 70)	**%16,47**	… normal load'lar çok daha fazla miss ediyor
`spill_st_L1miss`	6 / (54 + 6)	%10,00	Bütün stack yazmaları
`spill_event_L1miss`	3 / (42 + 3)	%6,67	Geri okunan stack yazmaları
`unread_st_L1miss`	(6 − 3) / (60 − 45)	%20,00	Hiç okunmayanlar daha çok miss ediyor
`other_st_L1miss`	(18 − 6) / (120 − 60)	%20,00	Spill olmayan store'lar
`L1D_miss`	48 / 360	%13,33	Bütün L1D access'leri
`L2_local_miss`	15 / 60	%25,00	Bütün L2 access'leri
`spill_ld_L2local`	1 / (1 + 1)	%50,00	Payda 2: yorumlamak için çok az
`unread_st_L2local`	(2 − 0) / (6 − 3)	%66,67	Payda 3: yorumlamak için çok az

Okuması:

- Spill load'lar L1D'de çok az miss ediyor: diğer load'lardaki %16,47'ye karşı %2,86. Yani spill trafiği cache dostu; stack'e yazılan değer geri okunurken genelde hâlâ L1'de.
- Hiç okunmayan spill store'lar (%20) okunanlardan (%6,67) daha çok miss ediyor. L2 local oranları ise bu kadar küçük paydalarla yorumlanmamalı.
- Kontrol: $45 \le 60 \le 120$, $45 \le 70 \le 240$, $54+6 = 60$ ve $4+2 = 6$ tutuyor. gem5'in L1D'de `stackStoreHits` = 54, `stackStoreMisses` = 6, `spillLoadHits` = 68, `spillLoadMisses` = 2; L2'de 4, 2, 1 ve 1 göstermesi gerekir.

C. Gerçek veriden iki uç

Test girdisi, small-2way konfigürasyonu:

- **cactuBSSN:** `spill/roi` %33,48, `spill_event/spill_store` %99,36. Komutların üçte biri spill-like ve stack'e yazılan neredeyse her şey geri okunuyor; register baskısı çok yüksek.
- **parest:** `spill/roi` %2,13. Spill trafiği az.